嵌入式系統設計
期末報告

影像辨識自走車

指導教授：陳慶瀚

組員：

114522007 李文甫

114522008 簡文勝

114522014 林暐智

目錄

一、 摘要.....2
二、 原理與方法.....2
三、 硬體架構.....3
四、 系統設計.....5
1. IDEF0.....5
2. Grafcet.....6
3. 軟體合成.....10
4. 硬體驗證.....20
五、 未來展望.....22
六、 附錄.....23
七、 參考資料.....33

一、 摘要

1.前言

隨著 AI 與 Embedded Systems 技術的快速發展，具備環境感知與自主決策能力的自走車是智慧交通、物流配送與服務型機器人等領域的重要研究方向。傳統以遙控方式操作的小車，在複雜或動態環境中，仍高度依賴人為操作，缺乏即時判斷與自動化能力，限制了其實際應用價值。

此專題結合影像辨識技術與嵌入式控制平台，設計並實作一套影像辨識自走車系統。系統透過攝影模組即時擷取環境影像，並將影像資料傳送至運算端進行物體辨識與判斷，藉此分析前方路況或目標物位置，進而生成對應的控制指令，使車輛能夠自主調整行進方向與行為。

在系統架構上，本專題以 ESP32 作為自走車的核心控制器，負責感測資料接收、馬達控制與無線通訊；影像辨識部分則利用已訓練完成的深度學習模型進行物件判斷，降低嵌入式裝置端的運算負擔。透過 WiFi 通訊機制，系統能即時完成影像傳輸與控制決策，實現軟硬體整合的影像辨識自走車。

二、 原理與方法

1. 影像擷取與 WiFi 傳輸

本系統利用攝影模組即時擷取自走車前方環境影像，並由 ESP32 作為主要控制核心，負責影像資料的初步處理與傳輸。由於嵌入式系統在運算能力與記憶體資源上有限，影像資料會以固定解析度與格式進行封裝後，透過 ESP32 內建的 WiFi 模組傳送至外部運算端。此設計能有效降低嵌入式端的處理負擔，同時維持影像資料的即時傳輸能力，作為後續影像辨識與控制決策的基礎。

2. 影像辨識 — YOLO v5 模型

影像辨識部分採用 YOLOv5 深度學習模型進行物件偵測。系統在接收到影像後，會先進行必要的前處理，再將影像輸入至 YOLOv5 模型中進行推論。模型能即時輸出目標物的類別與位置資訊，使系統能快速判斷環境狀態。YOLOv5 具備即時性佳與辨識準確度高的特性，適合應用於需要即時反應的自走車系統。

3. 馬達控制

自走車的運動由雙直流馬達驅動，透過馬達控制模組調整左右輪的轉動方向與

速度。ESP32 依據 Server 回傳的控制指令，輸出對應的 GPIO 訊號，以控制馬達前進、後退、轉向或停止。藉由改變左右輪的轉速差，系統能實現基本的車輛運動行為，並將影像辨識結果即時反映至實際行動。

4. 系統整合與即時性考量

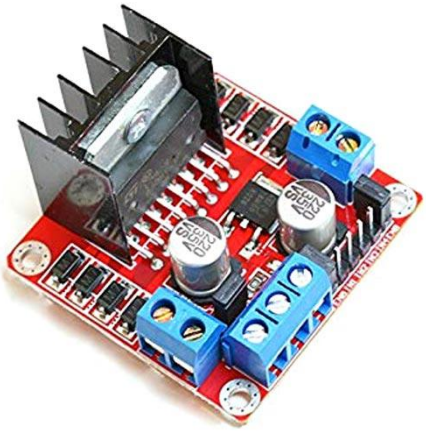
在系統整合過程中，本專題特別考量影像擷取、WiFi 傳輸、影像辨識與馬達控制之間的時間延遲。透過適當的影像解析度設定、通訊流程設計與控制邏輯簡化，使系統能在可接受的延遲範圍內完成「感知、判斷、行動」的循環。此整體設計確保自走車在實際運作時具備基本即時反應能力，並能穩定執行影像辨識導向的自走功能。

三、 硬體架構

1. ESP32-CAM + ESP-CAM-MB

2. OV3660 相機模組

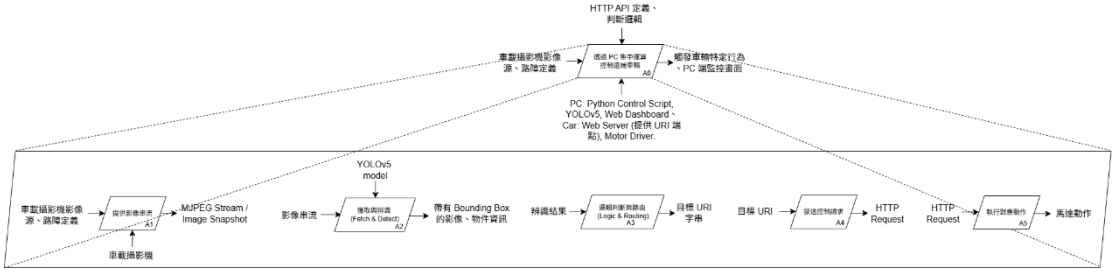
3. L289N 馬達控制模組

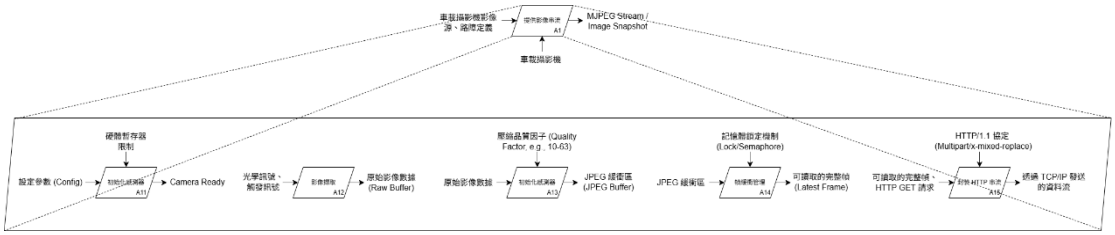


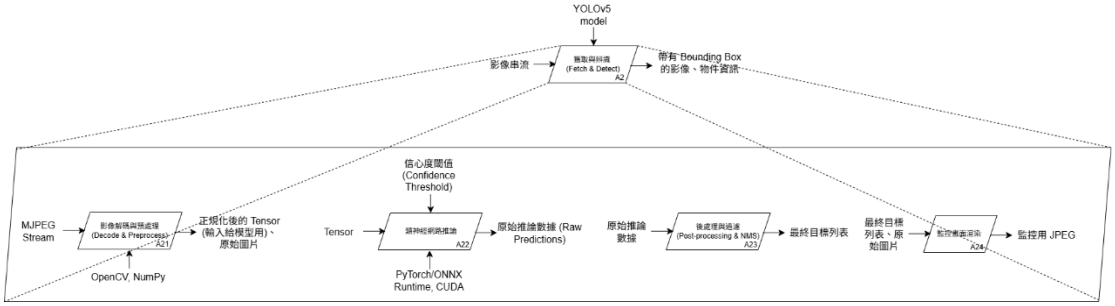
4. Robotica Tech MR2

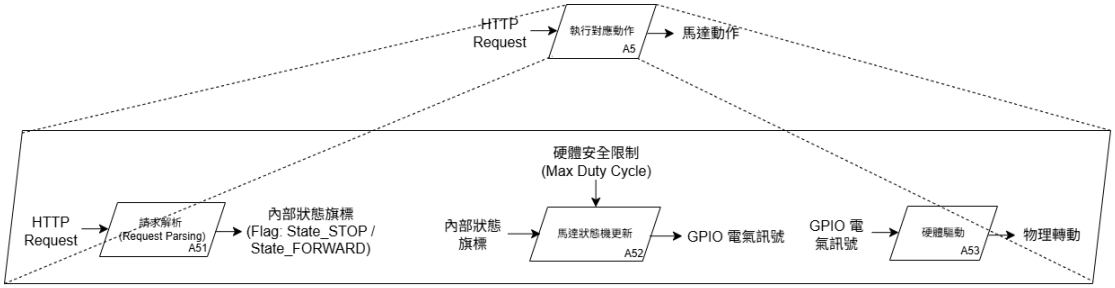
四、 系統設計

1. IDEF0



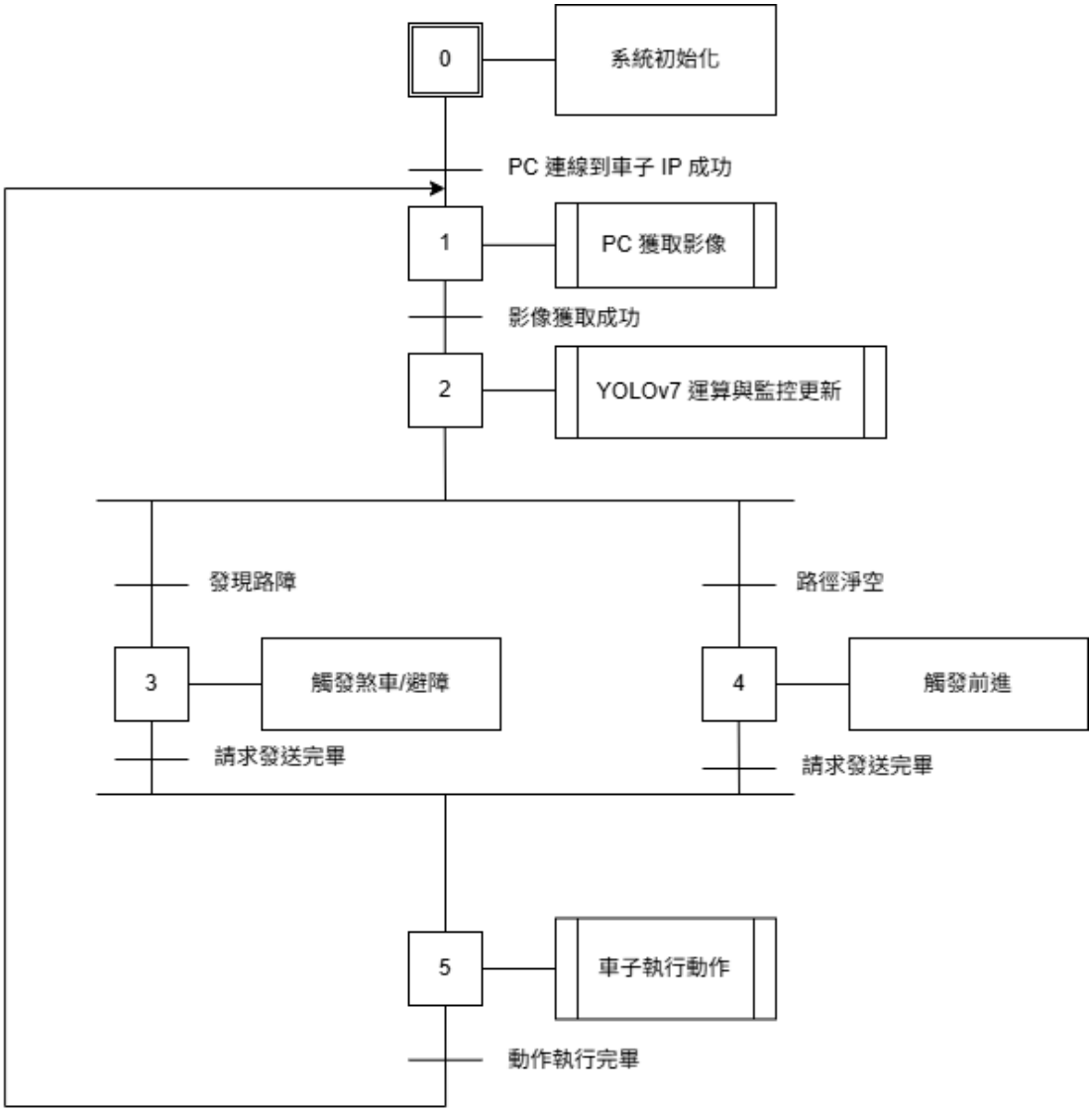




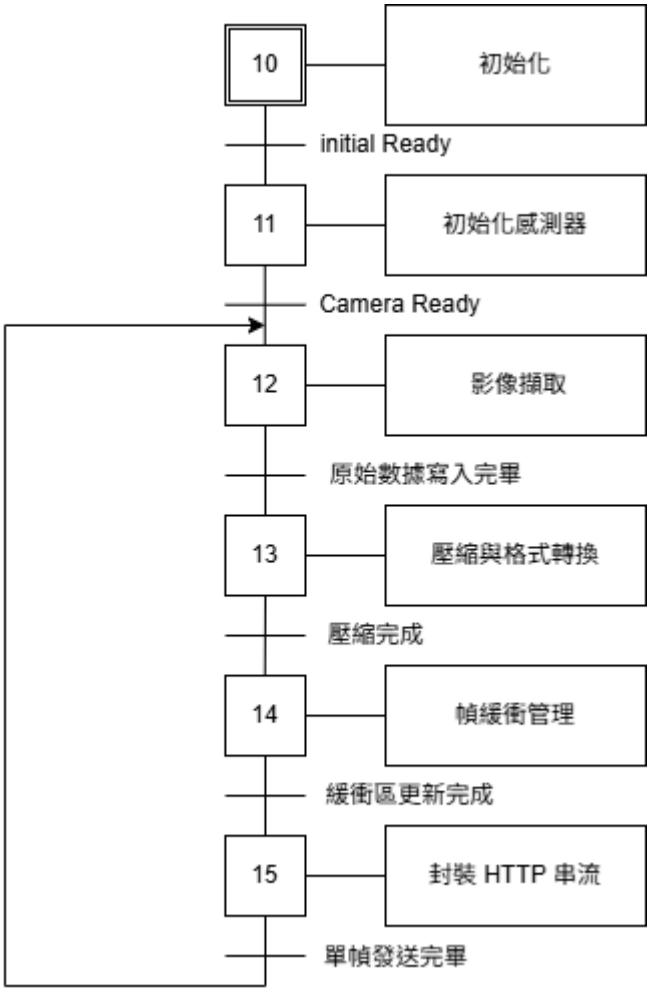


2. Grafcet

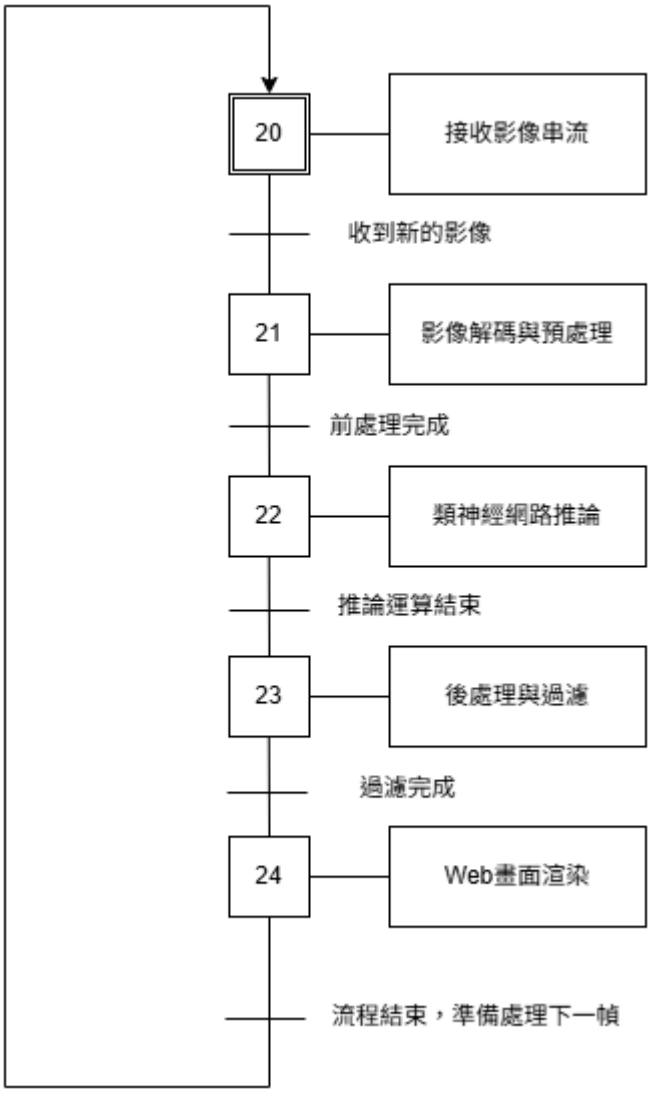
- Grafcet0



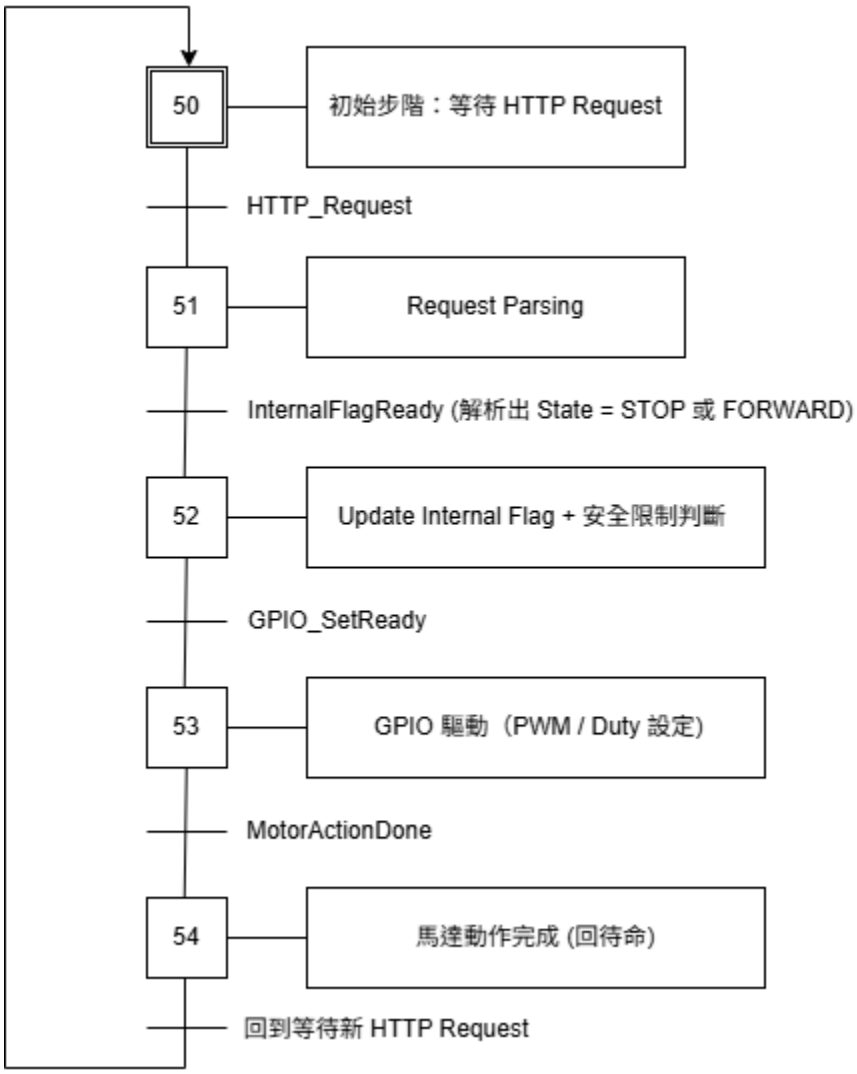
- Grafcet1


10
初始化
initial Ready
11
初始化感測器
Camera Ready
12
影像擷取
原始數據寫入完畢
13
壓縮與格式轉換
壓縮完成
14
幀緩衝管理
緩衝區更新完成
15
封裝 HTTP 串流
單幀發送完畢

- Grafcet2


20
接收影像串流
收到新的影像
21
影像解碼與預處理
前處理完成
22
類神經網路推論
推論運算結束
23
後處理與過濾
過濾完成
24
Web畫面渲染
流程結束，準備處理下一幀

- Grafcet5


50
初始步階：等待 HTTP Request
HTTP_Request
51
Request Parsing
InternalFlagReady (解析出 State = STOP 或 FORWARD)
52
Update Internal Flag + 安全限制判斷
GPIO_SetReady
53
GPIO 驅動（PWM / Duty 設定）
MotorActionDone
54
馬達動作完成 (回待命)
回到等待新 HTTP Request

3. 軟體合成

A. Arduino code 合成

```
// ======================= 共用資料結構 =======================

enum class Command {
    STOP,
    FORWARD
};

struct Frame {
    int         id;
    std::string mjpeg_stream;
};

struct YoloResult {
    int frame_id;
    std::vector<std::string> detections;

    bool has_obstacle() const {
        for (auto& d : detections) if (d == "obstacle") return true;
        return false;
    }
    bool has_stop_sign() const {
        for (auto& d : detections) if (d == "stop_sign") return true;
        return false;
    }
    bool path_clear() const {
        return !has_obstacle() && !has_stop_sign();
    }
};

struct RouteDecision {
    std::string uri;
    Command     cmd;
};
```

```
struct HttpRequest {
    std::string raw_text;
    Command     cmd;
};

// ======================= Grafcet 狀態位元 =======================

// A0: 系統控制 (Step 0~5)
int x0, x1, x2, x3, x4, x5;

// A1: 影像擷取 (第二層)
int x10, x11, x12, x13, x14, x15;     // x10: A1 start, x11~x15: A11~A15

// A2: YOLOv5 pipeline
int x20, x21, x22, x23, x24;          // x20: A2 start, x21~x24: A21~A24

// A5: Motor HTTP Server
int x50, x51, x52, x53;               // x50: A5 start, x51~x53: A51~A53

// ======================= 共用資料 =======================

int           g_frame_id = 0;
Frame         g_lastFrame;
YoloResult    g_lastYolo;
RouteDecision g_lastRoute;
HttpRequest   g_lastReq;

// 子系統完成旗標 (給 A0_transition 用)
bool g_A1_done = false;
bool g_A2_done = false;
bool g_A3_done = false;
bool g_A4_done = false;
bool g_A5_done = false;
```

```cpp
// ======================= 工具：印出 state vector ==================

void print_state_vector() {
    std::cout
        << "x0="  << x0  << " x1="  << x1  << " x2="  << x2
        << " x3=" << x3  << " x4="  << x4  << " x5="  << x5
        << "  x10=" << x10 << " x11=" << x11 << " x12=" << x12
        << " x13=" << x13 << " x14=" << x14 << " x15=" << x15
        << "  x20=" << x20 << " x21=" << x21 << " x22=" << x22
        << " x23=" << x23 << " x24=" << x24
        << "  x50=" << x50 << " x51=" << x51 << " x52=" << x52
        << " x53=" << x53
        << "\n";
}

// ======================= A1: Camera (x10 + A11~A15) ==============

void A1_step(bool enable) {
    if (!enable) {
        x10 = x11 = x12 = x13 = x14 = x15 = 0;
        return;
    }

    // Transition
    if (x10==0 && x11==0 && x12==0 && x13==0 && x14==0 && x15==0) {
        x10 = 1;      // Grafcet 起始步驟
    } else if (x10==1) {
        x10 = 0; x11 = 1;
    } else if (x11==1) {
        x11 = 0; x12 = 1;
    } else if (x12==1) {
        x12 = 0; x13 = 1;
    } else if (x13==1) {
        x13 = 0; x14 = 1;
    } else if (x14==1) {
        x14 = 0; x15 = 1;
    } else if (x15==1) {
        x15 = 0;
        g_A1_done = true;  // A1 完成一輪
```

```
    // Action
    if (x10==1) {
        // A1 起始，這裡可以視為「Camera Ready」
    }
    if (x11==1) {
        ++g_frame_id;
    }
    if (x12==1) {
        g_lastFrame.id = g_frame_id;
        g_lastFrame.mjpeg_stream = "MJPEG-Frame-" + std::to_string(g_frame_id);
    }
    // x13 / x14 / x15：可視為壓縮、緩衝區更新、封裝 HTTP stream
}

// ======================= A2: YOLOv5 (x20 + A21~A24) =======================

void A2_step(bool enable) {
    if (!enable) {
        x20 = x21 = x22 = x23 = x24 = 0;
        return;
    }

    // Transition
    if (x20==0 && x21==0 && x22==0 && x23==0 && x24==0) {
        x20 = 1;
    } else if (x20==1) {
        x20 = 0; x21 = 1;
    } else if (x21==1) {
        x21 = 0; x22 = 1;
    } else if (x22==1) {
        x22 = 0; x23 = 1;
    } else if (x23==1) {
        x23 = 0; x24 = 1;
    } else if (x24==1) {
        x24 = 0;
        g_A2_done = true;
    }
```

```
    // Action
    if (x20==1) {
        // 起始：可視為「建立 tensor buffer」
    }
    if (x22==1) {
        g_lastYolo.frame_id = g_lastFrame.id;
        g_lastYolo.detections.clear();

        // Demo：用 frame_id%3 做三種情境
        if (g_frame_id % 3 == 1) {
            // clear
        } else if (g_frame_id % 3 == 2) {
            g_lastYolo.detections.push_back("obstacle");
        } else {
            g_lastYolo.detections.push_back("stop_sign");
        }
    }
    // x21 / x23 / x24：前處理、後處理、畫面渲染
}

// ======================= A3: Logic & Routing (用 x3 代表)  ===========

void A3_step(bool enable) {
    if (!enable) {
        g_A3_done = false;
        return;
    }

    // A3 就是一個單步 Grafcet：在第一次啟動時做事，下一輪 A0 看到完成旗標
    if (!g_A3_done) {
        g_lastRoute.cmd = Command::FORWARD;
        if (g_lastYolo.has_obstacle() || g_lastYolo.has_stop_sign()) {
            g_lastRoute.cmd = Command::STOP;
        }
        g_lastRoute.uri = (g_lastRoute.cmd == Command::STOP) ?
                          "/cmd/stop" : "/cmd/forward";
        g_A3_done = true;
    }
```

```
// ======================= A4: HTTP Request Sender (用 x4 代表)  =====

void A4_step(bool enable) {
    if (!enable) {
        g_A4_done = false;
        return;
    }

    if (!g_A4_done) {
        g_lastReq.cmd = g_lastRoute.cmd;
        g_lastReq.raw_text = "GET " + g_lastRoute.uri +
                                 " HTTP/1.1\r\nHost: car.local\r\n\r\n";
        g_A4_done = true;
    }
}

// ======================= A5: Motor HTTP Server (x50 + A51~A53) ====

void A5_step(bool enable) {
    if (!enable) {
        x50 = x51 = x52 = x53 = 0;
        return;
    }

    // Transition
    if (x50==0 && x51==0 && x52==0 && x53==0) {
        x50 = 1;
    } else if (x50==1) {
        x50 = 0; x51 = 1;
    } else if (x51==1) {
        x51 = 0; x52 = 1;
    } else if (x52==1) {
        x52 = 0; x53 = 1;
    } else if (x53==1) {
        x53 = 0;
        g_A5_done = true;
    }
```

```
// ======================== A0: Top-level Grafcet ========================

void A0_transition() {
    // Step0: init -> Step1
    if (x0 == 1) {
        x0 = 0; x1 = 1;
        g_A1_done = g_A2_done = g_A3_done = g_A4_done = g_A5_done = false;
        return;
    }

    // Step1: 等 A1 完成
    if (x1 == 1 && g_A1_done) {
        x1 = 0; x2 = 1;
        g_A2_done = false;
        return;
    }

    // Step2: 等 A2 完成
    if (x2 == 1 && g_A2_done) {
        x2 = 0; x3 = 1;
        g_A3_done = false;
        return;
    }

    // Step3: 等 A3 完成
    if (x3 == 1 && g_A3_done) {
        x3 = 0; x4 = 1;
        g_A4_done = false;
        return;
    }

    // Step4: 等 A4 完成
    if (x4 == 1 && g_A4_done) {
        x4 = 0; x5 = 1;
        g_A5_done = false;
        return;
    }
```

```
    // Step5: 等 A5 完成後回到 Step1
    if (x5 == 1 && g_A5_done) {
        x5 = 0; x1 = 1;
        g_A1_done = g_A2_done = g_A3_done = g_A4_done = g_A5_done = false;
        return;
    }
}

// 每個 cycle 一句描述
std::string describe_state() {
    if (x0 == 1) return "系統初始化 (A0)";

    if (x1 == 1) {
        if (x10==1) return "A1: 起始狀態 (Camera Ready)";
        if (x11==1) return "A1: 攝影機初始化 / 設定參數";
        if (x12==1) return "A1: 攝影機擷取影像";
        if (x13==1) return "A1: 影像壓縮與格式轉換";
        if (x14==1) return "A1: 緩衝區管理 (更新最新影像)";
        if (x15==1) return "A1: 封裝 MJPEG HTTP 串流";
        return "A1: 影像擷取流程中";
    }

    if (x2 == 1) {
        if (x20==1) return "A2: 起始狀態 (建立 Tensor 輸入)";
        if (x21==1) return "A2: 影像解碼與前處理";
        if (x22==1) return "A2: YOLOv5 物件偵測";
        if (x23==1) return "A2: 後處理與 NMS";
        if (x24==1) return "A2: 監控畫面渲染";
        return "A2: 影像辨識流程中";
    }

    if (x3 == 1) {
        return "A3: 依偵測結果決定 STOP / FORWARD";
    }

    if (x4 == 1) {
        return "A4: 組出 HTTP 控制封包並送往自走車";
    }
```

```
    if (x5 == 1) {
        if (x50==1) return "A5: 起始狀態 (等待 HTTP Request)";
        if (x51==1) return "A5: 解析控制請求";
        if (x52==1) return "A5: 更新馬達狀態與安全限制 (Max Duty)";
        if (x53==1) return "A5: 透過 GPIO 驅動馬達";
        return "A5: 馬達動作執行中";
    }

    return "Idle / 無有效狀態";
}

// ======================= main: 軟體模擬 =======================

int main() {
    // 初始化狀態
    x0 = 1;  x1 = x2 = x3 = x4 = x5 = 0;
    x10 = x11 = x12 = x13 = x14 = x15 = 0;
    x20 = x21 = x22 = x23 = x24 = 0;
    x50 = x51 = x52 = x53 = 0;

    std::cout << "=== Grafcet Software Simulation ===\n";

    for (int cycle = 0; cycle < 25; ++cycle) {
        // 1. Top-level 依「上一輪完成狀態」做狀態轉移
        A0_transition();

        // 2. 各子系統在各自的步驟內前進一次
        A1_step(x1==1);
        A2_step(x2==1);
        A3_step(x3==1);
        A4_step(x4==1);
        A5_step(x5==1);

        // 3. 列出這一輪的 state vector + 一行說明
        std::cout << "\n" << describe_state() << "\n";
        print_state_vector();
    }
```

```
    std::cout << "\nSimulation finished.\n";
    return 0;
}
```

B. 模擬狀態轉移

```
A1: 起始狀態 (Camera Ready)
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=1 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 攝影機初始化 / 設定參數
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=1 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 攝影機擷取影像
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=1 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 影像壓縮與格式轉換
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=1 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 緩衝區管理 (更新最新影像)
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=1 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 封裝 MJPEG HTTP 串流
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=1  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 影像擷取流程中
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

```
A2: 起始狀態 (建立 Tensor 輸入)
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=1 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A2: 影像解碼與前處理
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=1 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A2: YOLOv5 物件偵測
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=1 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A2: 後處理與 NMS
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=1 x24=0  x50=0 x51=0 x52=0 x53=0

A2: 監控畫面渲染
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=1  x50=0 x51=0 x52=0 x53=0

A2: 影像辨識流程中
x0=0 x1=0 x2=1 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

```
A3: 依偵測結果決定 STOP / FORWARD
x0=0 x1=0 x2=0 x3=1 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

```
A4: 組出 HTTP 控制封包並送往自走車
x0=0 x1=0 x2=0 x3=0 x4=1 x5=0  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

```
A5: 起始狀態 (等待 HTTP Request)
x0=0 x1=0 x2=0 x3=0 x4=0 x5=1  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=1 x51=0 x52=0 x53=0

A5: 解析控制請求
x0=0 x1=0 x2=0 x3=0 x4=0 x5=1  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=1 x52=0 x53=0

A5: 更新馬達狀態與安全限制 (Max Duty)
x0=0 x1=0 x2=0 x3=0 x4=0 x5=1  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=1 x53=0

A5: 透過 GPIO 驅動馬達
x0=0 x1=0 x2=0 x3=0 x4=0 x5=1  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=1

A5: 馬達動作執行中
x0=0 x1=0 x2=0 x3=0 x4=0 x5=1  x10=0 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

```
A1: 起始狀態 (Camera Ready)
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=1 x11=0 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 攝影機初始化 / 設定參數
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=1 x12=0 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 攝影機擷取影像
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=1 x13=0 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 影像壓縮與格式轉換
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=1 x14=0 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0

A1: 緩衝區管理 (更新最新影像)
x0=0 x1=1 x2=0 x3=0 x4=0 x5=0  x10=0 x11=0 x12=0 x13=0 x14=1 x15=0  x20=0 x21=0 x22=0 x23=0 x24=0  x50=0 x51=0 x52=0 x53=0
```

4.硬體驗證

A. 腳位對應(ESP32-CAM)

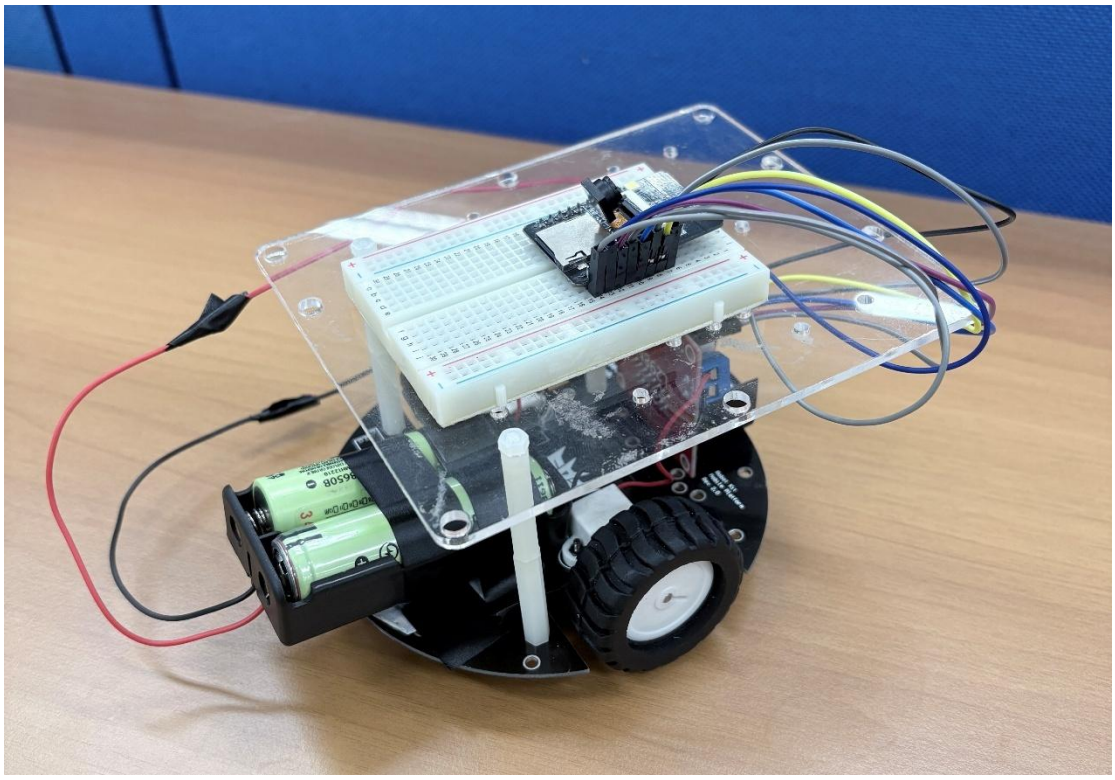
腳位名稱	ESP32 腳位編號	腳位類型	腳位描述
PWDN	GPIO 32	輸出	攝影機電源控制腳位，用於控制模組電源（高電位有效）
XCLK	GPIO 0	輸出	攝影機系統時脈，由 ESP32 提供外部時脈
SDA (SIOD)	GPIO 26	I/O	SCCB 資料線，用於設定攝影機暫存器
SCL (SIOC)	GPIO 27	輸出	SCCB 時脈線，用於設定攝影機暫存器
Y9	GPIO 35	輸出	影像資料位元 Y9
Y8	GPIO 34	輸出	影像資料位元 Y8
Y7	GPIO 39	輸出	影像資料位元 Y7
Y6	GPIO 36	輸出	影像資料位元 Y6
Y5	GPIO 21	輸出	影像資料位元 Y5
Y4	GPIO 19	輸出	影像資料位元 Y4
Y3	GPIO 18	輸出	影像資料位元 Y3

Y2	GPIO 5	輸出	影像資料位元 Y2
VSYNC	GPIO 25	輸出	垂直同步訊號，用於辨識影像一個畫面的開始與結束
HREF	GPIO 23	輸出	行同步訊號，用於辨識影像一行資料的有效區段
PCLK	GPIO 22	輸出	像素時脈，用於同步影像資料讀取

B. 腳位對應(馬達控制)

腳位名稱	ESP32 腳位編號	腳位類型	腳位描述
IN1	GPIO 2	輸出	左馬達方向控制腳位 1
IN2	GPIO 14	輸出	左馬達方向控制腳位 2
ENA	GPIO 4	輸出（PWM）	左馬達速度控制（PWM 訊號）
IN3	GPIO 15	輸出	右馬達方向控制腳位 1
IN4	GPIO 12	輸出	右馬達方向控制腳位 2
ENB	GPIO 13	輸出（PWM）	右馬達速度控制（PWM 訊號）

C. 實體電路



五、 未來展望

此專題可視為自動駕駛汽車之小型實驗模型，但是在實際道路環境中，影像畫面往往同時包含多個交通標誌、車輛或行人，而非單一目標。當多種標誌同時出現在畫面中，系統要有判斷行為優先順序的能力，如何在不同標誌之間做出正確或”相對”正確的決策，是一大挑戰。

未來可針對多目標同時辨識的情境，設計對應的決策規則或優先權機制，使系統行為更符合實際交通邏輯。未來也可進一步測試系統在不同光線、背景與通訊條件下的穩定性，並加強在影像遺失、辨識不確定或通訊延遲時的保護機制，以提升系統整體可靠度。

六、 附錄

提供影像串流_ESP-CAM 小車(A1)

```
#include "server.h"

#include "esp_camera.h"
#include <WiFi.h>
#include "esp_timer.h"
#include "img_converters.h"
#include "Arduino.h"
#include "fb_gfx.h"
#include "soc/soc.h" //disable brownout problems
#include "soc/rtc_cntl_reg.h"  //disable brownout problems
#include "esp_http_server.h"

// WIFI setting
const char* ssid = "HCILab_2.4G";
const char* password = "hcilab@307";

// OV3660 / ESP32-CAM 腳位
#define CAMERA_MODEL_AI_THINKER
#if defined(CAMERA_MODEL_AI_THINKER)
  #define PWDN_GPIO_NUM     32
  #define RESET_GPIO_NUM    -1
  #define XCLK_GPIO_NUM      0
  #define SIOD_GPIO_NUM     26
  #define SIOC_GPIO_NUM     27
  #define Y9_GPIO_NUM       35
  #define Y8_GPIO_NUM       34
  #define Y7_GPIO_NUM       39
  #define Y6_GPIO_NUM       36
  #define Y5_GPIO_NUM       21
  #define Y4_GPIO_NUM       19
  #define Y3_GPIO_NUM       18
  #define Y2_GPIO_NUM        5
  #define VSYNC_GPIO_NUM    25
  #define HREF_GPIO_NUM     23
  #define PCLK_GPIO_NUM     22
#endif

// 馬達腳位
#define PIN_IN1   2   // 左馬達方向 1
#define PIN_IN2  14   // 左馬達方向 2
#define PIN_ENA  4   // 左馬達 PWM
#define PIN_IN3  15   // 右馬達方向 1
#define PIN_IN4  12   // 右馬達方向 2
#define PIN_ENB  13    // 右馬達 PWM
```

```
void setup() {
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

  Serial.begin(115200);
  Serial.setDebugOutput(false);
  pinMode(PIN_IN1, OUTPUT);
  pinMode(PIN_IN2, OUTPUT);
  pinMode(PIN_ENA, OUTPUT);
  pinMode(PIN_IN3, OUTPUT);
  pinMode(PIN_IN4, OUTPUT);
  pinMode(PIN_ENB, OUTPUT);

  // 初始化，輪胎不動
  digitalWrite(PIN_IN1, LOW);
  digitalWrite(PIN_IN2, LOW);
  digitalWrite(PIN_IN3, LOW);
  digitalWrite(PIN_IN4, LOW);

  // ========================
  // OV3660 / ESP32-CAM 參數
  // ========================
  camera_config_t config;
  memset(&config, 0, sizeof(config));

  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;

  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;

  config.pin_xclk       = XCLK_GPIO_NUM;
  config.pin_pclk       = PCLK_GPIO_NUM;
  config.pin_vsync      = VSYNC_GPIO_NUM;
  config.pin_href       = HREF_GPIO_NUM;
  config.pin_sscb_sda   = SIOD_GPIO_NUM;
  config.pin_sscb_scl   = SIOC_GPIO_NUM;
  config.pin_pwdn       = PWDN_GPIO_NUM;
  config.pin_reset      = RESET_GPIO_NUM;
```

```
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_JPEG;

  if (psramFound()) {
    config.frame_size   = FRAMESIZE_VGA;  // 640x480
    config.jpeg_quality = 12;
    config.fb_count     = 2;
  } else {
    config.frame_size   = FRAMESIZE_QVGA; // 320x240
    config.jpeg_quality = 12;
    config.fb_count     = 1;
  }

  // 相機初始化
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x\n", err);
    return;
  }

  sensor_t * s = esp_camera_sensor_get();
  s->set_hmirror(s, 1);


  // Wi-Fi connection
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  Serial.print("Camera Capture Ready! Go to: http://");
  Serial.print(WiFi.localIP());

  // Start streaming web server
  startCameraServer();
}
```

```
void WheelAct(int IN1, int IN2, int ENA, int IN3, int IN4, int ENB){
  digitalWrite(PIN_IN1, IN1);
  digitalWrite(PIN_IN2, IN2);
  analogWrite(PIN_ENA, ENA);
  digitalWrite(PIN_IN3, IN3);
  digitalWrite(PIN_IN4, IN4);
  analogWrite(PIN_ENB, ENB);
}


const int cycle = 10;

void loop() {
  int parton = cycle * carSpeed/100;
  int lastSpeed = 0;
  switch (carDirection) {
    case FORWARD:
      WheelAct(HIGH , LOW , 209, HIGH, LOW, 185);
      break;
    case SPEED_LIMIT_30:
      WheelAct(HIGH , LOW , 145, HIGH, LOW, 125);
      break;
    case SPEED_LIMIT_120:
      WheelAct(HIGH , LOW , 255, HIGH, LOW, 247);
      break;
    default:
      // Stop the car
      WheelAct(LOW, LOW, 0, LOW, LOW, 0);
      break;
  }
  delay(parton);
  WheelAct(LOW, LOW, 0, LOW, LOW, 0);
  delay(cycle - parton);
}
```

獲取與辨識

```
img = fetch_img_from_car()

# return 偵測的標誌
objects_in_img = detect(img, img_counter, save_detection_img=True)
```

```python
def fetch_img_from_car():
    """
    從 ESP32-CAM 抓一張 JPEG → BGR 圖片
    """
    try:
        img = car.capture_img()   # 已是 BGR
        if img is None:
            raise RuntimeError("Failed to decode JPEG from ESP32-CAM")

        # === 修正圖片方向 ===
        img = cv.flip(img, 0)     # 上下翻轉
        img = cv.flip(img, 1)     # 左右翻轉

        return img

    except Exception as e:
        raise Exception(f"Error fetching image: {e}")
```

```python
def detect(img, img_counter, save_detection_img = True):
    img = img_handler.roi(img)

    # === 前處理 ===
    rgb_img = cv.cvtColor(img, cv.COLOR_BGR2RGB)
    modelResults = detection_model(rgb_img) # 這裡給出YOLO判斷結果

    objects_in_img, bboxes, colors, confidence = img_handler.extract_detection_info(modelResults)
    print(f"detection info: {objects_in_img,bboxes,colors,confidence}")
    return objects_in_img
```

邏輯判斷與路由_邏輯判斷(A3)

```python
# 設定標誌的優先順序並取得最高優先標誌
max_priority_obj, obj_size = get_max_priority_object(objects_in_img)
# 根據最高優先標誌調整車速
car_speed = drive(max_priority_obj, last_obj_in_image, obj_size, car_speed)
```

```python
# max_priority_object： 從檢測到的物體列表中返回優先級最高的物體
def get_max_priority_object(detected_objects : list):
    def get_priority_of_object(obj, obj_size):
        if obj == STOP_SIGN and obj_size > 400 :
            return 6
        elif obj == SPEED_LIMIT_30_SIGN and obj_size > 100:
            return 5
        elif obj == SPEED_LIMIT_120_SIGN and obj_size > 100 :
            return 4
        else:
            return -10 #

    max_priority = -1
    size = 0
    max_priority_obj = None
    for obj, obj_size in detected_objects:
            obj_priority = get_priority_of_object(obj, obj_size)
            if obj_priority > max_priority:
                max_priority = obj_priority
                size = obj_size
                max_priority_obj = obj
    return max_priority_obj,size
```

```python
# 定義駕駛邏輯，根據檢測到的物體調整車速和方向
def drive(max_priority_obj,last_obj_in_image, obj_size,car_speed):
    """
    根據當前最高優先物件，決定車子的行為 & 更新 car_speed
    回傳：更新後的 car_speed
    """
    if max_priority_obj == STOP_SIGN and last_obj_in_image != STOP_SIGN:
        car.stop()
        time.sleep(3)  # 停3秒
        car.move_forward()
        car_speed = INITIAL_SPEED
    elif max_priority_obj == SPEED_LIMIT_30_SIGN:
        car.speed_limit_30()
        car_speed = LIMIT_30_SPEED
    elif max_priority_obj == SPEED_LIMIT_120_SIGN:
        car.speed_limit_120()
        car_speed = LIMIT_120_SPEED
    else:
        car.move_forward()

    return car_speed
```

啟用 Wi
移至 [設定]

邏輯判斷與路由_路由部分(A3)

```python
# Car control URIs
CAR_ESP_32_URI = "YOUR_ESP-CAM_IP_ADDRESS"
PORT = 80

# URI設定
FORWARD_URI = f"{CAR_ESP_32_URI}:{PORT}/forward"
STOP_URI = f"{CAR_ESP_32_URI}:{PORT}/stop"
SPEED_LIMIT_30_URI = f"{CAR_ESP_32_URI}:{PORT}/speed/30"
SPEED_LIMIT_120_URI = f"{CAR_ESP_32_URI}:{PORT}/speed/30"
```

```python
################
# 車輛動作邏輯
################
def move_forward():
    try:
        urllib.request.urlopen(FORWARD_URI,timeout=2)
    except urllib.error.URLError as e:
        print(f"URLError occurred while sending the 'go forward' command:\n {e}")

def stop():
    try:
        urllib.request.urlopen(f"{STOP_URI}", timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while stopping the car:\n {e}")

def speed_limit_30():
    try:
        urllib.request.urlopen(SPEED_LIMIT_30_URI, timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while setting speed limit to 30:\n {e}")

def speed_limit_120():
    try:
        urllib.request.urlopen(SPEED_LIMIT_120_URI, timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while setting speed limit to 120:\n {e}")
```

啟用 Windows
移至 [設定] 以啟用 Windows

發送控制請求(A4)

```python
# Car control URIs
CAR_ESP_32_URI = "YOUR_ESP-CAM_IP_ADDRESS"
PORT = 80

# URI設定
FORWARD_URI = f"{CAR_ESP_32_URI}:{PORT}/forward"
STOP_URI = f"{CAR_ESP_32_URI}:{PORT}/stop"
SPEED_LIMIT_30_URI = f"{CAR_ESP_32_URI}:{PORT}/speed/30"
SPEED_LIMIT_120_URI = f"{CAR_ESP_32_URI}:{PORT}/speed/30"
```

```python
################
# 車輛動作邏輯
################
def move_forward():
    try:
        urllib.request.urlopen(FORWARD_URI,timeout=2)
    except urllib.error.URLError as e:
        print(f"URLError occurred while sending the 'go forward' command:\n {e}")

def stop():
    try:
        urllib.request.urlopen(f"{STOP_URI}", timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while stopping the car:\n {e}")

def speed_limit_30():
    try:
        urllib.request.urlopen(SPEED_LIMIT_30_URI, timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while setting speed limit to 30:\n {e}")

def speed_limit_120():
    try:
        urllib.request.urlopen(SPEED_LIMIT_120_URI, timeout=2)
    except urllib.error.URLError as e:
        print(f"URL error occurred while setting speed limit to 120:\n {e}")
```

啟用 Windows
移至 [設定] 以啟用 Windows

執行對應動作(A5)

```
void startCameraServer() {
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();
    config.server_port = 80;
    // 把上限改大
    config.max_uri_handlers = 16;

    // ====== API URI 定義 ======
    httpd_uri_t index_uri = {
        .uri       = "/",
        .method    = HTTP_GET,
        .handler   = index_handler,
        .user_ctx  = NULL
    };
    httpd_uri_t forward_uri = {
        .uri       = "/forward",
        .method    = HTTP_GET,
        .handler   = forward_handler,
        .user_ctx  = NULL
    };
    httpd_uri_t stop_uri = {
        .uri       = "/stop",
        .method    = HTTP_GET,
        .handler   = stop_handler,
        .user_ctx  = NULL
    };
    httpd_uri_t speed_limit_30_uri = {
        .uri       = "/speed/30",
        .method    = HTTP_GET,
        .handler   = speed_limit_30_handler,
        .user_ctx  = NULL
    };
    httpd_uri_t speed_limit_120_uri = {
        .uri       = "/speed/120",
        .method    = HTTP_GET,
        .handler   = speed_limit_120_handler,
        .user_ctx  = NULL
    };
```

```
// ====== 啟動 Web Server ======
if (httpd_start(&index_httpd, &config) == ESP_OK) {

    httpd_register_uri_handler(index_httpd, &index_uri);
    httpd_register_uri_handler(index_httpd, &forward_uri);
    httpd_register_uri_handler(index_httpd, &speed_limit_30_uri);
    httpd_register_uri_handler(index_httpd, &speed_limit_120_uri);
    httpd_register_uri_handler(index_httpd, &stop_uri);

    Serial.printf("HTTP server started on port 80\n");
}
```

辨識結果展示

七、 參考資料

1. 影像辨識自走車參考
https://github.com/YanivShahar1/Autonomous-Object-Detecting-Toy-Car?tab=readme-ov-file
2. 交通號誌辨識模型
https://github.com/MDhamani/Traffic-Sign-Recognition-Using-YOLO?tab=readme-ov-file
3. 課程講義 PPT